

Relatório Problema 2: Unidade Lógica Aritmética

Gustavo Silva Ribeiro, João Carlos Rodrigues Souza, Diego dos Santos Barros Santana

Universidade Estadual de Feira de Santana (UEFS)

Av. Transnordestina, s/n, Novo Horizonte, Feira de Santana – BA, Brasil – 44036-900

[tavoribeiro007@gmail.com, diegoe.comp@gmail.com, joaocarlosrodrigues307@gmail.com]

Resumo. *Este trabalho apresenta o projeto e a implementação de uma calculadora digital de 8 bits baseada em Notação Polonesa Reversa (RPN), desenvolvida na disciplina de Circuitos Digitais (MI) em Verilog para a FPGA DE10-Lite. O sistema expande o conceito de ULA para um módulo robusto de 8 bits, capaz de realizar operações aritméticas (soma, subtração, multiplicação com saturação e divisão com detecção de zero) e lógicas (AND, OR, XOR, NOT). O protótipo armazena automaticamente o último resultado e o exibe em displays de 7 segmentos nas bases decimal, octal e hexadecimal.*

1. Introdução

As Unidades Lógicas e Aritméticas (ULAs) constituem elementos fundamentais em processadores e microcontroladores, sendo responsáveis pela execução das operações aritméticas e lógicas essenciais. O estudo de ULAs, frequentemente iniciado com arquiteturas de 4 bits, é expandido neste projeto para um sistema de 8 bits, o que possibilita a exploração de funcionalidades mais avançadas, como operações complexas e uso de memória.

O objetivo deste projeto é projetar e implementar uma calculadora digital baseada em Notação Polonesa Reversa (RPN) em uma plataforma FPGA (placa DE10-Lite). O sistema proposto utiliza operandos de 8 bits e foi desenvolvido em Verilog, com foco em circuitos sequenciais e combinacionais.

A calculadora implementada suporta a entrada de dados no modelo RPN e executa um conjunto de operações aritméticas, lógicas e especiais. As operações aritméticas incluem soma ($A+B$), subtração ($A-B$), multiplicação ($A \times B$) com saturação e divisão ($A \div B$) com detecção de divisão por zero. As operações lógicas implementadas são AND, OR, XOR e NOT.

Entre os requisitos funcionais, o sistema possui um registrador de memória para o armazenamento automático do último resultado e sinaliza o estado da operação através de flags, como Overflow (OV), Zero (Z), Carry Out (COUT) e Erro (ERR). A visualização dos resultados é realizada em displays de 7 segmentos, com a funcionalidade de seleção da base de exibição (hexadecimal, decimal e octal) por meio de chaves da placa.

O desenvolvimento deste protótipo visa capacitar os discentes na compreensão de conceitos avançados de ULAs e pilhas digitais. O projeto exige a aplicação de boas práticas de modularização e documentação em Verilog, bem como a capacidade de analisar, depurar e testar operações complexas diretamente no hardware do FPGA. O produto final esperado é um protótipo funcional acompanhado de um relatório técnico com a fundamentação teórica, simulações e testes práticos.

Este relatório detalha a especificação do circuito, a implementação da estrutura em Verilog na ferramenta Quartus e as estruturas de testes e simulações utilizadas para a validação do protótipo.

2. Requisitos e funcionalidades

Esta seção detalha os requisitos e especificações para o projeto da calculadora aritmética e lógica baseada em ULA.

Funcionalidades do Sistema

- **Arquitetura:** O sistema deve operar com operandos de 8 bits.
- **Modelo de Entrada:** A calculadora deve implementar a lógica de Notação Polonesa Reversa (RPN) para a entrada e execução de operações.
- **Operações Aritméticas:** A ULA deve ser capaz de realizar Soma ($A + B$), Subtração ($A - B$), Multiplicação ($A \times B$) e Divisão ($A \div B$).
- **Operações Lógicas:** A ULA deve implementar as operações bit-a-bit AND ($A \& B$), OR ($A | B$), XOR ($A \wedge B$) e NOT ($\sim A$).
- **Memória:** O último resultado válido deve ser armazenado automaticamente em um registrador para ser reutilizado como operando em cálculos futuros.
- **Flags de Estado:** O sistema deve gerar flags para indicar o estado da última operação:
 - **Overflow (OV):** Indica que o resultado de uma subtração gerou um bit de borrow out.
 - **Zero (Z):** Indica que o resultado é zero.
 - **Carry out (COUT):** Indica que a soma gerou um bit de carry-out.
 - **Erro (ERR):** Indica uma operação inválida ou divisão por 0.
- **Tratamento de Erro:** O sistema deve detectar e sinalizar (via flag ERR) qualquer tentativa de divisão por zero.
- **Conversão de Base:** O resultado deve poder ser convertido e exibido nas bases Decimal, Hexadecimal e Octal.

Interface de Hardware e Periféricos

- **Plataforma Alvo:** O projeto será sintetizado e implementado na placa FPGA DE10-Lite.
- **Entradas de Dados (RPN):** Botões (push-buttons) da placa serão usados para a entrada de dados e execução de operações RPN.
- **Seleção de Base:** Chaves (switches) da placa serão usadas para selecionar a base de exibição do resultado.
- **Exibição de Resultado:** Os displays de 7 segmentos da placa devem mostrar o resultado numérico.
- **Exibição de Flags:** LEDs da placa devem indicar o estado das flags (OV, Z, COUT, ERR).

Restrições de Implementação e Validação

- **Linguagem:** O projeto deve ser implementado exclusivamente em Verilog, utilizando um estilo estrutural.
- **Ferramenta:** O software Quartus deve ser utilizado para a síntese.
- **Restrição da Multiplicação:** O módulo de multiplicação deve ser implementado recursivamente, utilizando apenas um somador, e deve incluir lógica de saturação.
- **Otimização:** O projeto final deve corresponder ao circuito mínimo necessário para atender aos requisitos.

2.1. Metodologia

2.1.1 Circuitos Combinacionais

Circuitos combinacionais são aqueles cujas saídas são determinadas exclusivamente pela combinação de suas entradas atuais. Eles não possuem memória de estados anteriores e são a base para o processamento lógico e aritmético [Tocci, 2011].

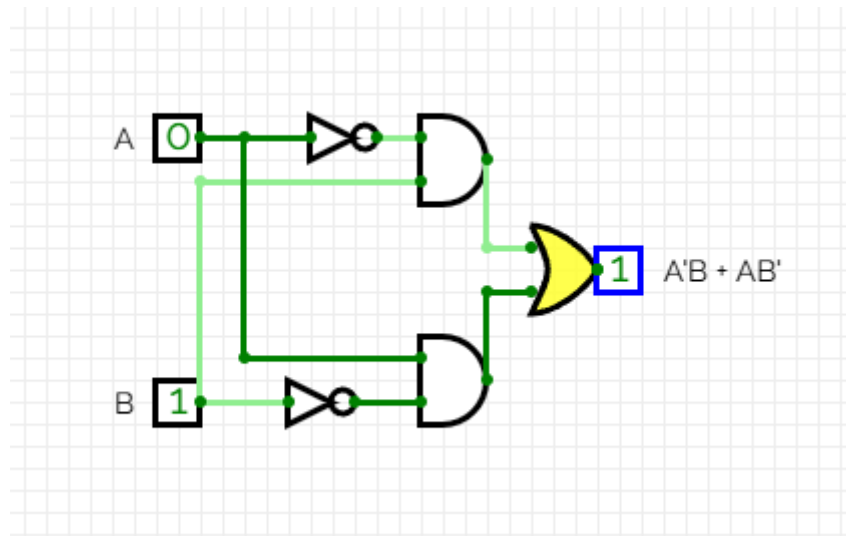


Figura 1. Um exemplo de um Circuito Combinacional, onde as saídas dependem exclusivamente das entradas.

Fonte: Autoria própria

2.1.2 Circuitos Sequenciais

Circuitos sequenciais, em contrapartida, são aqueles cujas saídas dependem não apenas das entradas atuais, mas também do estado anterior do circuito, sendo esta a característica fundamental que os dota de "memória". Este estado anterior é fisicamente armazenado em dispositivos de memória fundamentais, como latches ou flip-flops, os quais são projetados especificamente para reter um bit de informação (0 ou 1) até que uma nova transição seja permitida.

A transição do estado desses bits é controlada por um sinal de clock, que atua sincronizando as transições de estado de todos os elementos de memória simultaneamente.

Essa sincronização é crucial, pois garante que o próximo estado seja calculado e armazenado apenas em instantes discretos e bem definidos, tipicamente na borda de subida do pulso de clock, assegurando um comportamento previsível e estável, permitindo assim a implementação de lógicas complexas como contadores, registradores e máquinas de estado finito (FSMs).

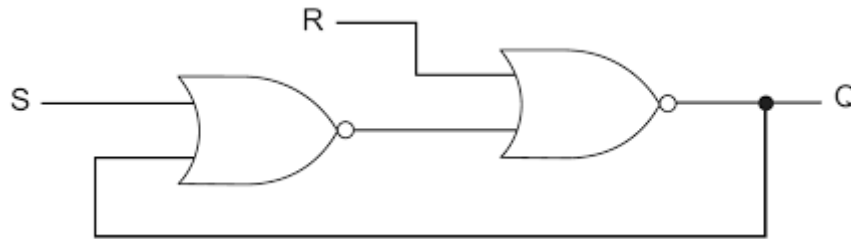


Figura 2.Um exemplo de um Circuito Sequencial, onde as saídas atuais influenciam diretamente nas próximas saídas
Fonte: Moreno, E.

2.1.2.1 Flip-flops.

Os flip-flops são pequenos componentes do circuito que atuam como uma memória digital, sendo capazes de armazenar até 1 bit de informação. Esse bit é alterado apenas durante as transições do sinal de clock inserido, ou seja, no instante exato em que o sinal muda de nível baixo para alto, ou de alto para baixo.

Entre os tipos principais de flip-flops, temos os Flip-flops S-R, os quais são flip-flops que possuem três entradas, sendo elas S (Set), R (Reset) e CLK (Clock), cujo estado é alterado (com base nas entradas S e R) apenas na transição de nível baixo para o nível alto (0 para 1) do sinal de clock, porém, esse flip-flop possui a desvantagem de que caso a entrada S seja igual a entrada R e R seja igual a 1, o circuito chega em uma ambiguidade, uma vez que, esse comando acaba por ordenar que a saída seja definida e 1, e resetada simultaneamente, gerando uma saída imprecisa e não confiável.

Além dele, temos o Flip-Flop J-K, que também possui três entradas (J, K e CLK) e funciona de maneira muito similar ao S-R, onde J atua como a entrada Set e K atua como a entrada Reset. A sua grande vantagem é que ele resolve o problema do estado de ambiguidade do S-R (quando S e R são 1 ao mesmo tempo). No Flip-Flop J-K, quando as entradas J e K estão ambas em nível alto (1) durante a borda do clock, o flip-flop não entra em um estado indefinido; em vez disso, ele executa a operação 'Toggle', a qual inverte o sinal de sua saída.

E por fim, temos o Flip-flop tipo D, o escolhido para a criação do projeto, que é um flip-flop que é acionado no momento de subida do clock (de 0 para 1), mas, diferentemente dos outros modelos, possui apenas 1 única entrada, o D (‘data’, pois armazenará um dado), a função deste flip-flop é a mais direta, no exato instante da borda de subida do clock, a saída ‘Q’ copia e armazena o valor (0 ou 1) que estiver presente na entrada D. E, por ter apenas uma entrada que define diretamente o próximo estado, ele elimina por completo a possibilidade de qualquer ambiguidade ou estado proibido, servindo como uma célula de

memória pura e confiável, e esse atributo que o confere confiabilidade e principalmente, em decorrência de sua simplicidade em termos de número de entradas, saídas e operações, esse tipo de Flip-Flop se tornou atrativo e foi então o escolhido para o desenvolvimento do projeto.

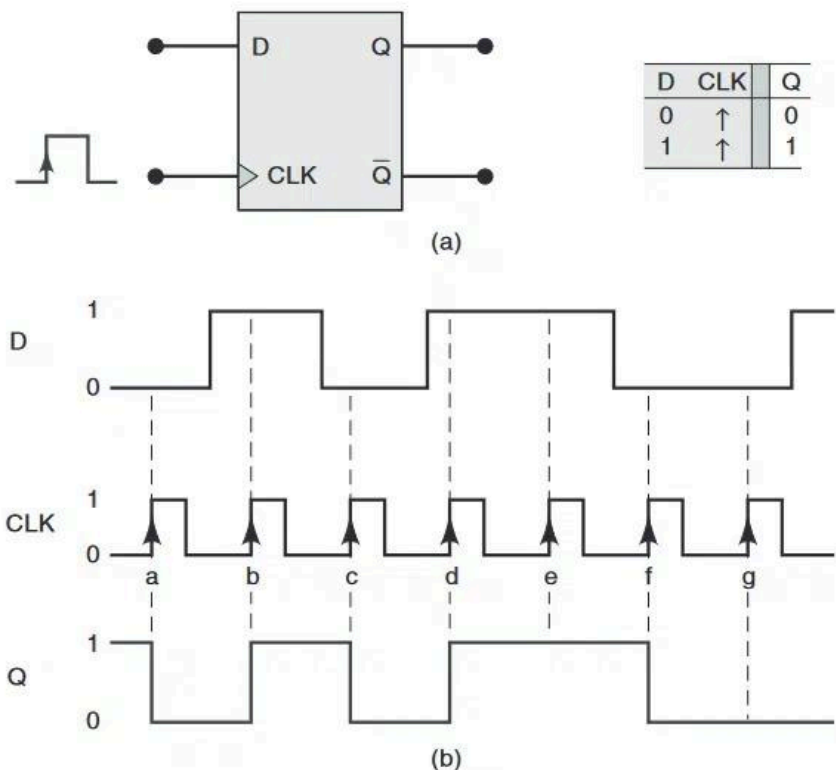


Figura 3. Flip-flop do tipo D
Fonte: Embarcados

2.1.2.2 Registradores.

A fim de criar algum circuito capaz de armazenar múltiplos bits em sequência, optamos pela utilização de registradores, que são um grupo de flip-flops que armazenam múltiplos bits ao mesmo tempo e nos quais todos os bits do valor binário armazenado estão diretamente disponíveis[Tocci, 2011].

E para isso, utilizamos uma sequência de flip-flops do tipo D, onde cada Flip-Flop D é responsável por armazenar um único bit da sequência. O elemento crucial que une essa sequência é o sinal de clock comum, que é conectado à entrada CLK de todos os flip-flops simultaneamente, permitindo que todos se atualizem simultaneamente, preservando os dados e reduzindo as chances de erros consideravelmente.

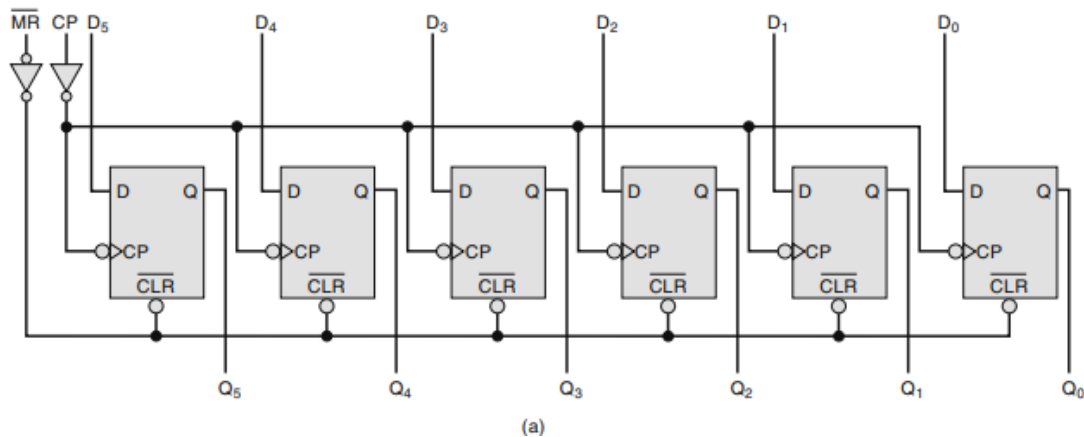


Figura 4. Circuito registrador com entradas e saídas paralelas
Fonte: (Tocci, 2011, Pag. 379, FIGURA 7.67)

2.1.2.3 Contadores.

Os contadores são circuitos digitais que variam os seus estados, sob o comando de um clock, de acordo com uma sequência predeterminada [Idoeta, 2012]. Esses circuitos são majoritariamente utilizados principalmente para realizar contagens, divisão de frequência e entre outras tarefas, e no geral, podem ser classificados de duas maneiras, de acordo com a forma de ativação dos flip flops que os compõem, como assíncronos e síncronos.

2.1.2.4 Contadores assíncronos.

Esses contadores recebem esse nome em decorrência dos seus flip-flops, uma vez que eles operam de maneira assíncrona, onde a entrada do clock é inserida apenas no primeiro flip-flop do circuito, e o clock de todos os outros flip-flops ocorrem em decorrência da saída dos blocos anteriores [Idoeta, 2012].

No nosso projeto, este contador foi escolhido para servir como um ponto de parada para o circuito multiplicador, para que este saiba quando é o momento de parar de somar o valor inserido com ele mesmo, e ele foi escolhido em decorrência de sua simplicidade e fácil implementação, nos permitindo terminar o circuito do multiplicador e realizar os devidos testes.

2.1.2.5 Contadores Síncronos

Os contadores síncronos são contadores cujas entradas de clock são curto-circuitadas, ou seja, o clock entra em todos os flip-flops simultaneamente, fazendo todos atuarem de forma sincronizada.

No projeto, este contador foi utilizado na entrada de dados para que o circuito possa manter noção de qual dado está sendo inserido nesse momento, nos permitindo saber qual é a primeira entrada binária, qual a segunda entrada binária e qual a operação a ser executada com precisão e sem risco de entradas incorretas.

2.1.2.6 Algoritmo Double Dabble

Para exibir os resultados obtidos no display de 7 segmentos, precisávamos de um método eficiente e confiável para converter os valores binários para o sistema decimal e quaisquer outras bases a serem utilizadas, e para isso, utilizamos o algoritmo Double-Dabble, um algoritmo responsável por converter um número binário na notação Binary-Coded Decimal(BCD), a qual separa o valor binário em grupos de 4 bits onde cada grupo representa uma casa decimal do valor e pode assumir valores de 1 a 9.

Esse algoritmo é implementado seguindo uma série de passos. Inicialmente, verifica-se se um conjunto de 4 bits é maior do que 5, em caso afirmativo, soma-se 3 ao conjunto de 4 bits, e desloca o valor para a esquerda. Caso não seja maior do que 5, o valor apenas é deslocado para a esquerda. Após uma série de verificações, uma palavra binária é convertida em grupos de 4 bits em formato BCD.

2.1.2.7 Reverse Polish Notation (RPN)

Notação Polonesa Reversa (Reverse polish notation), é um método de escrita de expressões matemáticas em que os operadores são posicionados após os seus respectivos operandos, simulando uma espécie de “pilha” de valores, sendo eles, o valor 1, o valor 2 e a operação. Essa abordagem, também conhecida como notação pós-fixada, foi adotada no projeto em decorrência de sua simplicidade, uma vez que com ela, não haveria necessidade de se levar em conta a ordem de operações, e ao invés de priorizar operações em parênteses e multiplicações/divisões, poderíamos focar apenas na operação a ser realizada entre as duas últimas entradas.

No circuito, essa notação foi implementada de modo que, os 2 primeiros valores inseridos seriam os operandos A, e B, e o terceiro valor inserido, seria a operação a ser realizada, desse modo, a calculadora realiza a operação inserida, entre o valor A, e o valor B, com A sendo o primeiro valor na operação, e B o segundo, e o resultado, tem a possibilidade de se tornar o valor A da próxima operação, ou ser descartado, permitindo uma melhor usabilidade da calculadora.

3. Desenvolvimento

O protótipo do circuito foi desenvolvido em uma FPGA (Field Programmable Gate Array) da família MAX 10, modelo 10MDAF484C7G, disponível no Laboratório de Eletrônica Digital e Sistemas (LEDS) da Universidade Estadual de Feira de Santana. A placa de desenvolvimento utilizada é a DE10-Lite (Figura 5). Todo o projeto foi descrito em Verilog Estrutural e sintetizado utilizando o software Quartus Prime Lite Edition.

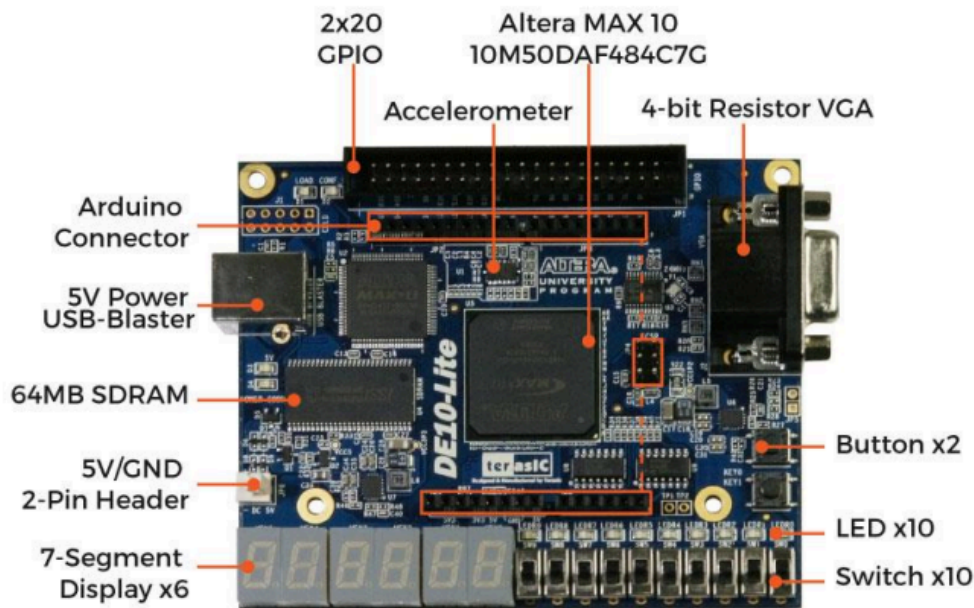


Figura 5. Visão Geral da placa DE10-Lite
Fonte: DE10 - Lite User Manual

O desenvolvimento do protótipo baseou-se na combinação de módulos lógicos, contendo a associação de portas lógicas e componentes digitais necessários para a implementação das funções desejadas. Inicialmente, foi realizada a decomposição do problema em blocos funcionais menores, permitindo identificar as entradas, saídas e sinais intermediários de cada módulo. Em seguida, cada bloco foi descrito em linguagem Verilog HDL, possibilitando a simulação individual dos componentes antes da integração completa do sistema.

A estrutura geral do circuito foi projetada de forma modular, de modo que cada unidade possua uma função bem definida como controle, processamento ou exibição e possa ser facilmente testada ou substituída. Os periféricos de entrada, como chaves (switches) e botões de controle, foram utilizados para fornecer sinais binários ao sistema, enquanto os LEDs e displays de sete segmentos atuaram como dispositivos de saída, permitindo a visualização do comportamento lógico do circuito durante os testes práticos na FPGA.

As entradas foram pensadas para reproduzir o formato de entrada de dados RPN (Reverse Polish Notation), de modo que 8 chaves (switches) são usadas para informar o número binário ou operação a ser realizada, 1 chave é responsável por carregar essa informação em uma pilha digital, e 1 chave é utilizada para inserir o resultado da última operação novamente na pilha. Já os botões foram configurados para alternar entre diferentes modos de exibição, possibilitando visualizar o resultado em bases decimal, hexadecimal e octal, conforme a escolha do usuário.

Além disso, o protótipo é sincronizado com uma entrada de clock de 50 MHz nativo da FPGA DE10-Lite. Nesse sentido, os principais componentes da placa DE10-Lite usados para realizar a entrada de dados foram 10 chaves, 2 botões e uma entrada de clock (Tabela 1).

Sinal de Entrada	Função
SW0	Bit de dado / operação
SW1	Bit de dado / operação
SW2	Bit de dado / operação
SW3	Bit de dado
SW4	Bit de dado
SW5	Bit de dado
SW6	Bit de dado
SW7	Bit de dado
SW8	Envia número à pilha
SW9	Envia resultado à pilha
KEY0	Seletor da base numérica de exibição
KEY1	Seletor da base numérica de exibição
MAX10CLK150	Clock de 50 MHz

Tabela 1. Tabela de correspondência entre sinais de entrada e suas funções
Fonte: Autoria própria

As saídas do circuito foram projetadas para exibir o resultado das operações em quatro displays de sete segmentos, permitindo a visualização dos valores em diferentes bases numéricas. Além disso, seis LEDs adicionais foram empregados para sinalizar informações complementares sobre o estado do sistema, como a ocorrência de resultados que ultrapassam 12 bits — limite de representação do protótipo em todas as bases — ou a detecção de condições de erro. Dessa forma, quando a entrada de dados é finalizada, os displays apresentam os resultados das operações, enquanto os LEDs fornecem feedback visual imediato sobre a confiabilidade e o correto funcionamento do circuito (Tabela 2).

Sinal de saída	Descrição
Display 1 (HEX00 - HEX06)	Imprime primeiro número do resultado
Display 2 (HEX10 - HEX16)	Imprime segundo número do resultado
Display 3 (HEX20 - HEX26)	Imprime terceiro número do resultado
Display 4 (HEX30 - HEX36)	Imprime quarto número do resultado
LEDR0	Houve Carry out na soma
LEDR1	Houve Borrow out na subtração
LEDR2	Houve erro na divisão
LEDR3	Resultado da operação foi igual a zero
LEDR4	Resultado armazenado da última operação é impreciso
LEDR5	Resultado da operação excedeu 12 bits

Tabela 2. Tabela de correspondência entre os sinais de saída e o que representam
Fonte: Autoria própria

A lógica de entrada de dados no protótipo ocorre através das 10 chaves. Inicialmente, um número é inserido utilizando as oito primeiras chaves, representando o valor binário a ser processado. Em seguida, a chave designada para carregar o número na pilha digital é acionada, armazenando-o para posterior operação. Para inserir um segundo número, o mesmo procedimento é repetido: o usuário configura o valor nas oito chaves e pressiona novamente a

chave de carregamento. Após a entrada dos números, um valor correspondente à operação a ser realizada é inserido nas três primeiras chaves, e a chave de processamento é acionada. Por fim, a última chave, responsável por reinserir o resultado da operação na pilha, permite que o sistema realize cálculos sequenciais de forma contínua, mantendo a lógica de uma pilha fundamentada na lógica RPN.

4. Descrição funcional dos módulos do protótipo

A nível de desenvolvimento técnico, o código em linguagem de descrição de hardware foi construído de forma modular, com cada módulo desempenhando funções específicas dentro do sistema a partir de suas entradas e saídas. O circuito principal é formado pela junção de vários módulos que podem ser agrupados em seis grupos principais: Entrada de dados baseada em RPN, operações aritméticas e lógicas, flags, registrador do último resultado, representação do resultado em displays de 7 segmentos e o módulo principal.

4.1. Entrada de dados

Para implementar a lógica de entrada de dados utilizando a notação polonesa inversa no circuito foram utilizados dois módulos principais: *switch_debouncer* e *shift_register*. Estes, por sua vez, foram agrupados em um módulo denominado top, responsável por integrar a lógica de funcionamento do *switch_debouncer* e do *shift_register*, garantindo que ambos funcionem conjuntamente.

4.1.1 Switch debouncer

O *switch_debouncer* foi projetado para receber a entrada que sinaliza que uma informação deve ser carregada na pilha e garantir que apenas um pulso seja enviado para o *shift_register*. Esse módulo é composto por 2 flip-flops do tipo D e uma instância do módulo *divisor_frequencia*.

O módulo *divisor_frequencia* é composto por 6 instâncias de 3 flip-flops conectados em cascata. Em cada ciclo de clock, o primeiro flip-flop muda seu estado a cada pulso de clock, enquanto o flip-flop seguinte é acionado pela negação da saída do primeiro. O mesmo acontece para o terceiro flip-flop. Esses flip-flops são resetados quando o primeiro e o segundo flip-flop têm suas saídas em 1, e enviam, ao mesmo tempo, a saída do segundo flip-flop, um pulso com frequência 5 vezes menor, para o próximo grupo de 3 flip-flops, que irão dividir a frequência recebida em 5, novamente. Esse processo se repete 6 vezes.

Depois que o sinal do clock é reduzido por estas 6 instâncias, ele passa por uma série de flip-flops que dividem a frequência mais 2 vezes e a enviam para a saída *clk_botao*. Essa saída é utilizada no módulo *switch_debouncer* para verificar a estabilidade do sinal proveniente da chave (switch) utilizada para adicionar números na pilha digital. A verificação acontece através de 2 flip-flops do tipo D que recebem o sinal da chave e mudam seu estado a partir do clock enviado pelo módulo *divisor_frequencia*. A cada clock enviado, o primeiro flip-flop (*sync1*) guarda o sinal recebido da chave, e após outro ciclo, esse sinal é enviado para o segundo flip-flop (*stable*) que tem como saída um sinal limpo (*botao_ok*) que pode ser usado para operar a pilha digital do circuito, garantindo que um sinal só seja enviado caso a informação proveniente da chave esteja estável.

4.1.2 Shift register

O módulo *shift_register* é a estrutura responsável por criar a lógica que replica uma pilha digital. Ele é composto por 3 instâncias de um registrador de 8 bits, um contador síncrono e um flip-flop. Os registradores são módulos compostos por 8 flip-flops do tipo D que recebem, cada um, um bit de uma palavra de 8 bits e mudam seu estado quando recebem um sinal de clock, armazenando, dessa forma, uma informação, que é enviada para a saída de 8 bits denominada *out*.

Os registradores de 8 bits são conectados em sequência de modo que a informação armazenada em um registrador seja “passada” para o registrador seguinte. A entrada do primeiro registrador (*in*) é a palavra de 8 bits proveniente das chaves. Enquanto, o segundo registrador recebe, como entrada, saída do primeiro, e o terceiro, a saída do segundo. Além disso, os registradores só mudam de estado com um pulso de clock (*en*). Dessa forma, a cada pulso de clock o primeiro registrador recebe a informação vinda das chaves, e os outros registradores recebem a informação armazenada no registrador anterior.

O *contador_mod4* é um módulo projetado para implementar um contador síncrono de 2 bits que realiza a contagem sequencial de 0 a 3. Este módulo é composto por 2 flip-flops do tipo D, os flip-flops operam com um sinal de enable (*en*) que controla quando a contagem deve avançar. Quando o sinal *en* está ativo, ambos os flip-flops são acionados simultaneamente.

O flip-flop (*dff0*), responsável pelo bit menos significativo *out[0]*, possui sua entrada D conectada à saída de uma porta NOT que inverte o valor atual de *out[0]*. Isso faz com que este bit alterne seu estado a cada pulso de clock. O flip-flop (*dff1*), responsável pelo bit mais significativo *out[1]*, possui sua entrada D conectada à saída de uma porta XOR que atua sobre os valores atuais de *out[1]* e *out[0]*. Esta configuração faz com que o bit mais significativo mude de estado apenas quando o bit menos significativo estiver em nível lógico 1.

A sequência de contagem segue o padrão: 00 (0) → 01 (1) → 10 (2) → 11 (3) → 00 (0), completando um ciclo a cada 4 pulsos de clock. Ambos os flip-flops permanecem com reset desativado (1'b0) durante toda a operação, garantindo que a contagem seja controlada exclusivamente pela lógica implementada pelas portas NOT e XOR.

Este contador é utilizado para gerar uma sequência de controle e garantir que um sinal seja enviado para apresentar o resultado das operações aritméticas somente quando o contador chegar a 10 (2). Esse sinal, por sua vez, é armazenado em um flip-flop que opera com um sinal de enable (*en*) e tem como saída a porta denominada *executar*. Dessa forma, no terceiro pulso de enable (*en*), quando o contador chegar em 11 (3), o sinal *executar* permanecerá ativo até que o usuário acione a chave de controle mais uma vez.

4.1.3 Top

O módulo *top* agrupa os módulos *switch_debouncer* e *shift_register*, funcionando como a unidade de controle principal que gerencia o carregamento e armazenamento de dados na pilha digital. Esse módulo tem como entrada as portas *clk*, *switch_in* e *data_in* que representam, respectivamente, o clock nativo da FPGA, a chave que carrega uma informação na pilha e as 8 chaves que representam um número ou a seleção da operação. E, como saída,

este módulo envia as informações armazenadas em cada registrador ('out1', 'out2' e 'out3') e um sinal que sinaliza que a operação deve ser realizada, chamado de *execute*.

Quando a chave *switch_in* é pressionada, o sinal com bouncing é enviado para o *switch_debouncer*, que filtra as instabilidades mecânicas e gera um pulso limpo (*pulse_en*) com duração de um ciclo de clock. Este pulso seguro é então encaminhado para o *shift_register* como sinal de enable.

No *shift_register*, o sinal *pulse_en* desencadeia duas operações principais: primeiro, o valor presente nas chaves *data_in* é carregado no primeiro registrador da pilha (*out1*); segundo, os valores previamente armazenados são deslocados para os registradores seguintes, seguindo a sequência *out1* → *out2* → *out3*. Dessa forma, a pilha mantém sempre os três últimos valores inseridos, com *out1* representando o dado mais recente e *out3* o mais antigo.

A saída *execute* é ativada quando o *shift_register* detecta que o contador chegou a 11 (3). Esta sinalização indica que o sistema deve realizar uma operação aritmética ou lógica usando os valores armazenados em *out1*, *out2* e *out3* como operandos.

4.2. Operações aritméticas e lógicas

O circuito desenvolvido realiza um conjunto de operações, incluindo aritméticas (soma, subtração, multiplicação e divisão) e as lógicas (AND, OR, XOR e NOT). Os módulos responsáveis por implementar essas operações são chamados, respectivamente, de *sum8bit*, *sub8bit_correto*, *multiplicador_alternativo*, *divisor8bit*, *AND_operation*, *OR_operation*, *XOR_operation*, *NOT_operation*.

4.2.1 Somador de 8 bits

O somador de 8 bits foi implementado através da cascata de oito módulos *sum* de 1 bit (Figura 6), cada um responsável por realizar a operação de soma completa (full adder) entre bits individuais.

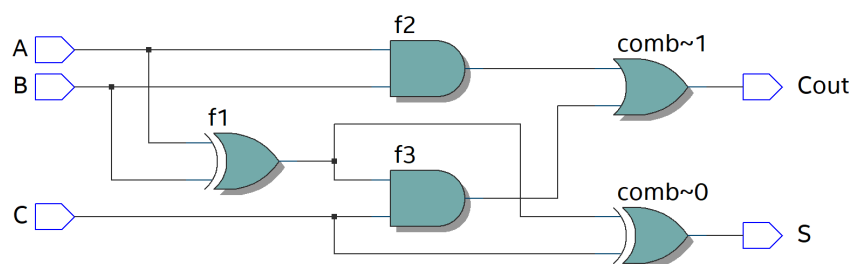


Figura 6. Implementação do módulo *sum*
Fonte: Autoria própria

Cada módulo *sum* emprega lógica combinacional construída com portas AND, OR e XOR para processar as entradas *A*, *B* e *Carry-in* (*C*), gerando como saídas o bit de resultado *S* e o *Carry-out* (*Cout*). A estrutura do somador segue o princípio de propagação do carry, onde a saída *Cout* de cada estágio é conectada à entrada *C* do estágio subsequente, formando uma

cadeia de transporte de vai-um. O primeiro somador tem seu *carry-in* inicializado com 0, garantindo que a operação inicie sem *carry* prévio. Esta estrutura permite a extensão da capacidade de soma para 8 bits, onde cada módulo processa um par de bits correspondentes das palavras de entrada, enquanto propaga o *carry* de forma sequencial através de toda a cadeia de somadores.

4.2.2 Subtrator de 8 bits

O subtrator de 8 bits foi implementado através da conexão em cascata de oito módulos *sub* de 1 bit, cada um responsável por realizar a subtração completa entre 2 bits (*A* e *B*).

Cada módulo *sub* emprega lógica combinacional construída com portas NOT, AND, XOR e OR para processar as entradas *A*, *B* e *Borrow-in* (*bin*), gerando como saídas o bit de resultado *S* e o *Borrow-out* (*bout*). A estrutura do subtrator segue o princípio de propagação de *borrow*, onde a saída *bout* de cada subtrator é conectada à entrada *bin* do subtrator seguinte, formando uma cadeia de transporte de "empresta-um".

A lógica do módulo *sub* opera da seguinte forma: primeiramente, calcula-se o XOR entre *B* e *bin* (*f3*), que representa a diferença parcial. Em seguida, o resultado final *S* é obtido através do XOR entre *A* e *f3*. Para o *borrow-out*, são considerados dois casos: quando *B* e *bin* são ambos 1 (*f1*), ou quando *A* é 0 e a diferença parcial *f3* é 1 (*f2*). O *borrow-out* *bout* é a operação OR entre esses dois casos (Figura 8).

O primeiro subtrator tem seu *borrow-in* inicializado com o valor *bin* 0. Através da conexão de 8 módulos *sub* em cascata, a capacidade de subtração é estendida para 8 bits, com cada módulo *sub* processando a subtração entre um par de bits, enquanto propaga o *borrow*, através da cadeia de subtratores. Além disso, o circuito verifica se houve *borrow-out* no último subtrator da sequência para garantir que o resultado da subtração é positivo. Isso é feito através de um AND entre todas as saídas *S* dos subtratores individuais e a negação do *borrow-out* (*nbout*) do último subtrator.

Desse modo, caso exista um *borrow-out*, o resultado da subtração será 0 e, caso não haja, o resultado normal será enviado ao display.

4.2.3 Multiplicador de 8 bits

O módulo *multiplicador alternativo* foi desenvolvido para realizar a multiplicação entre dois números de 8 bits (*A* e *B*) por meio de um processo baseado em somas sucessivas. Sua implementação é composta por blocos modulares que permitem a execução da operação de forma síncrona, controlada pelo sinal de clock (*clk*) e pelo sinal de início (*start*).

O funcionamento do circuito é baseado na lógica de repetição de somas: a cada ciclo de clock, o valor de *A* é somado ao acumulador (*acc*) até que o contador alcance o valor de *B*, representando a quantidade de vezes que *A* deve ser adicionado a si mesmo. Dessa forma, ao término do processo, o registrador de 16 bits armazena o produto resultante da multiplicação.

O controle do número de iterações é feito pelo módulo *contador_mod255*, responsável por contar de 0 até 255. Esse contador é ativado pelo sinal *cont_enable* e reiniciado sempre que *reset* é acionado. Para determinar o momento de finalização da multiplicação, o módulo *comparador_8bit* compara continuamente o valor atual do contador

(count) com o operando *B*. Quando ambos são iguais, o sinal *count_done* é ativado, indicando que todas as somas necessárias foram realizadas.

A cada iteração, a soma é executada pelo módulo *sum16bit*, que realiza a operação entre o acumulador (acc) e o valor de *A* expandido para 16 bits. Esse somador é formado pela conexão em série de 16 módulos *sum*.

O resultado parcial é armazenado no registrador de 16 bits (register_16bit), composto pela junção de dois registradores de 8 bits (register_8bit). O sinal de *enable* (enable_acc) garante que novos valores sejam carregados no acumulador apenas enquanto o processo estiver ativo (sinal start habilitado e contador ainda não finalizado).

Ao término da operação, o sinal *done* é ativado, permitindo que os bits do acumulador sejam transferidos para a saída *out*, representando o resultado final da multiplicação (Figura 7).

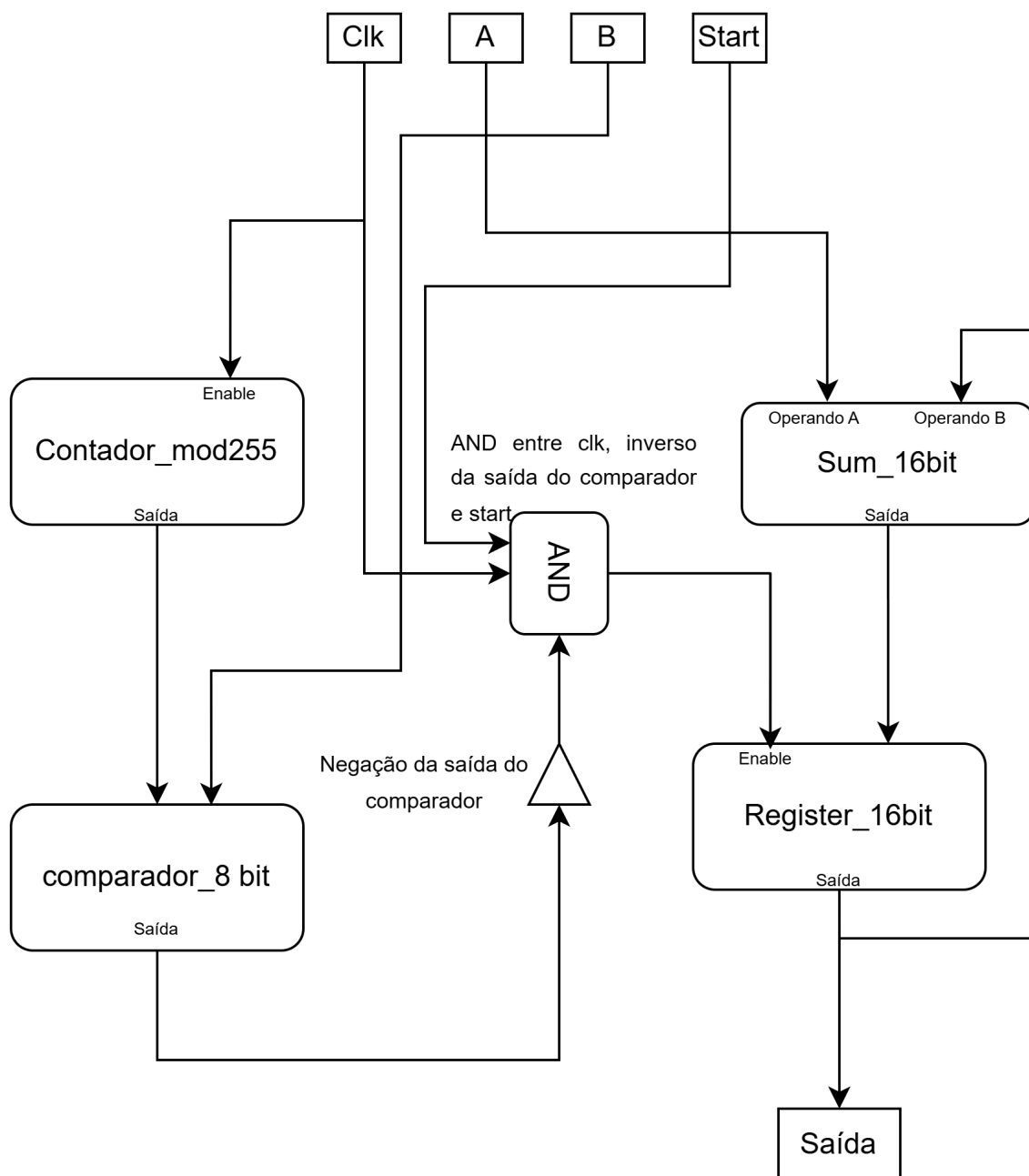


Figura 7. Diagrama de funcionamento do multiplicador binário de 8 bits
Fonte: Autoria própria

4.2.4 Divisor de 8 bits

O divisor binário foi construído através de um processo de subtrações em sequência e da verificação do *borrow out* das subtrações. O módulo *divisor8bit*, onde o circuito foi implementado, também conta com um sistema de detecção de divisão por zero, garantindo que o resultado seja corretamente anulado caso o divisor (B) seja igual a zero.

A estrutura do divisor é composta por oito subtratores de 8 bits (*sub8bit*) conectados em sequência e por módulos multiplexadores (*mux_divisor*), que controlam a escolha entre o resultado da subtração e o valor anterior, com base no sinal de *borrow-out* (*bout*). Esse *borrow* indica se o dividendo parcial é menor que o divisor, determinando se o quociente parcial deve receber um bit 0 ou 1.

Durante o processo de divisão, o módulo *sub8bit* realiza a subtração entre o dividendo parcial e o divisor. O resultado obtido é então enviado ao *mux_divisor*, que, com base no sinal de *borrow-out*, seleciona se o próximo dividendo será o valor subtraído (quando não ocorre *borrow*) ou o valor original antes da subtração (quando ocorre *borrow*). O bit correspondente do quociente (*Qtemp*) é definido pela negação desse *borrow-out*, de modo que, se a subtração for bem-sucedida (sem *borrow*), o bit do quociente assume o valor 1; caso contrário, assume o valor 0.

O *mux_divisor* é composto por oito multiplexadores 2x1, responsáveis por selecionar entre os bits correspondentes dos vetores de entrada A e B conforme o sinal de controle *sel*. No início da operação, o circuito verifica se o divisor (B) é igual a zero, utilizando portas OR e NOT para identificar essa condição. Quando a divisão por zero é detectada, o sinal *notdivzero* é desativado e o resultado Q é definido como zero por meio de portas AND, evitando um resultado inválido.

4.2.5 Operações lógicas

Os módulos responsáveis por executar as operações lógicas entre duas palavras de 8 bits ou apenas uma palavra de 8 bits são: *AND_operation*, *OR_operation*, *XOR_operation*, *NOT_operation*. Todos, exceto *NOT_operation*, são compostos por duas entradas de 8 bits (A, B) e uma saída de 8 bits (S).

Os módulos *AND_operation*, *OR_operation*, *XOR_operation* processam duas palavras de 8 bits de entrada (A e B) e geram uma palavra de 8 bits na saída (S). Cada um desses módulos é composto por oito portas lógicas idênticas operando em paralelo, onde cada porta realiza operações bit a bit entre os bits correspondentes das palavras de entrada.

No módulo *AND_operation* cada par de bits (*A[i]*, *B[i]*) é submetido a uma porta AND, gerando o bit de saída *S[i]* correspondente. A mesma lógica se aplica aos módulos OR e XOR, que utilizam portas OR e XOR, respectivamente, para processar cada posição bit a bit. Já o módulo *NOT_operation*, processa apenas uma palavra de 8 bits. Sua implementação consiste em 8 portas NOT independentes, onde cada porta inverte um bit da palavra de entrada, gerando como resultado o complemento bit a bit da palavra original.

4.3. Flags

O protótipo construído apresenta um sistema dedicado para sinalizar situações que possam afetar o resultado das operações através dos leds da FPGA DE10-Lite. Os circuitos responsáveis por executar essa função são chamados: *overflow*, *overflow12bits*, *overflow8bits*, *FlagErro*, *CarryOut* e *FlagZero*.

4.3.1 Overflow

O módulo *overflow* foi projetado para verificar se uma operação de subtração ocorreu corretamente. Ele utiliza uma porta lógica AND para gerar um sinal de saída sempre que ocorre *Borrow-out*. Essa saída está conectada ao LEDR1, de forma que, quando o resultado da subtração não puder ser representado corretamente, o LED acenderá, indicando a ocorrência de Borrow-out.

Os módulos *overflow8bits* e *overflow12bits* têm a função de sinalizar situações específicas relacionadas ao resultado da última operação. Ambos recebem como entrada um valor de 15 bits denominado *Saída*, que corresponde ao resultado da operação armazenado no registrador ou exibido no display. O *overflow8bits* verifica se esse resultado excede 8 bits, enquanto o *overflow12bits* verifica se excede 12 bits. Devido à limitação do registrador e ao fato de que as entradas das operações possuem 8 bits, um resultado maior que isso pode gerar imprecisão ao ser reutilizado. Além disso, a capacidade máxima de representação do circuito nas bases numéricas decimal, hexadecimal e binária é de 12 bits. Por esse motivo, esses módulos alertam o usuário quando os resultados podem não estar corretos.

Esses módulos verificam a presença de bits em posições específicas da entrada *Saída* usando uma porta lógica OR. O *overflow8bits* analisa os bits de posição 8 a 15, enquanto o *overflow12bits* analisa os bits de posição 12 a 15. Sempre que algum desses bits estiver em nível lógico alto, o módulo retorna um sinal positivo, indicando a ocorrência de *overflow*, e acendendo o LEDR4 (*overflow8bits*) ou o LEDR5 (*overflow12bits*).

4.3.2 Flag de Erro

O módulo *FlagErro* tem como finalidade sinalizar situações em que, durante uma operação de divisão, o divisor é igual a zero, indicando que ocorreu um erro na divisão dos números.

O funcionamento do módulo é baseado nas suas entradas *B*, de 8 bits, e *O*, de 3 bits, e gera um sinal de saída *S*. Internamente, todos os bits da entrada *B* passam por uma porta lógica NOR. Dessa forma, se todos os bits de *B* forem iguais a zero, a porta retorna 1.

Além disso, o módulo verifica se a operação executada é uma divisão por meio da entrada *O*. Essa verificação é feita através de uma porta AND que combina a negação do bit 0 de *O*, os bits 2 e 1 de *O* e a saída da porta NOR aplicada à entrada *B*. Com essa lógica, o módulo consegue identificar quando ocorre um erro ao tentar dividir dois números, sinalizando-o na saída *S*, que acende o LEDR2.

4.3.3 Flag de Carry out

O módulo *CarryOut* foi desenvolvido para sinalizar quando, ao realizar uma soma, o resultado excede 8 bits. O funcionamento desse circuito acontece através de uma porta AND

entre a entrada *Cout*, que retorna um sinal para a saída *S*. A saída desse circuito é conectada ao LEDR0 que acende quando *S* é 1.

4.3.2 Flag Zero

O circuito responsável por verificar se o resultado é igual a zero é chamado de *FlagZero*, que tem uma entrada de 12 bits chamada *S* e uma saída de 1 bit denominada *SI*. Esse módulo desempenha sua função através de uma porta lógica NOR aplicada aos 12 bits de entrada e com sua saída conectada a *SI*. Dessa forma, quando todos os bits de entrada forem 0, *SI* assume nível lógico alto e acendendo o LEDR3

4.4. Registrador do último resultado

A parte do protótipo responsável por registrar o resultado da última operação é composta pela instância de 3 flip-flops do tipo D, uma porta AND e um registrador de 8 bits. Os flip-flops estão conectados em sequência, de modo que o primeiro recebe um sinal de clock, e os flip-flops seguintes recebem a negação da saída do flip-flop anterior. A cada ciclo de clock, um pulso mais lento é gerado por esses flip-flops.

A porta AND é utilizada para gerar o pulso que emite um sinal de *enable* para o registrador, indicando que o resultado deve ser armazenado. Esse AND é aplicado ao sinal proveniente dos flip-flops e ao sinal *executar* emitido quando 3 valores foram adicionados à pilha digital e a operação foi realizada.

Por fim, o registrador de 8 bits recebe o resultado da operação aritmética proveniente do multiplexador principal, o sinal de *enable* da porta AND e tem como saída a porta *last_result* que armazena o valor da última operação até que uma nova operação seja realizada.

4.5. Display de 7 segmentos

Para permitir a representação do resultado das operações aritméticas em diferentes bases decimais, foram construídos 3 decodificadores, um para a base decimal, um para a hexadecimal e um a para octal, que convertem bits em uma palavra que representa um número para ser apresentado nos displays. Através da junção desses 3 decodificadores em único módulo que implementa uma lógica de seleção entre as 3 bases, foi criado um sistema que altera a base mostrada no display.

4.5.1 Decodificador decimal

O módulo *display_decimal* foi desenvolvido para agrupar os módulos que implementam a lógica de conversão de uma palavra binária de 12 bits para 3 grupos de 4 bits em formato decimal codificado em binário (BCD), que são então enviados para 3 instâncias do módulo *decoder_decimal* que enviam informações para os displays, apresentando um número decimal.

A lógica de conversão para BCD foi implementada através da técnica chamada *double_dabble*. Essencialmente, esse algoritmo verifica-se um conjunto de 4 bits é maior do que 5, em decimal, em caso afirmativo, adicionando 3 e deslocando todos os bits a serem

convertidos em uma posição para a esquerda, e, em caso negativo, apenas deslocando. Após realizar essa operação para todos os bits, o circuito fornece uma informação codificada em BCD (Figura 8).

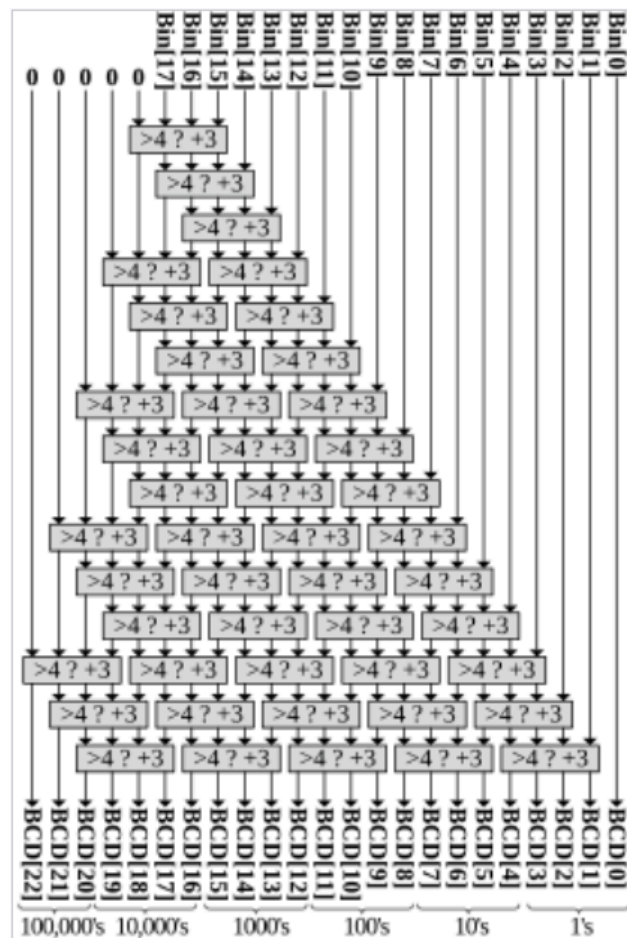


Figura 8. Implementação paramétrica do double dabble binário para BCD, para 18 bits

Fonte: Abdelhadi, Ameer

O módulo *double_dabble* foi projetado para implementar a técnica de conversão binário para BCD. Dentro do módulo, a verificação é feita pela instância de *eh_maior5*, que analisa os quatro bits de entrada e retorna um sinal de controle. Esse sinal determina se o valor é maior que 5, ativando a adição de 3 por meio do somador de 4 bits.

O resultado dessa soma é então direcionado aos quatro multiplexadores 2x1 (mux2x1), que escolhem entre manter o valor original ou utilizar o valor somado, de acordo com o sinal retornado pelo módulo *eh_maior5*. Assim, quando o valor é menor ou igual a 5, os multiplexadores mantêm os bits originais; quando é maior que 5, os bits correspondentes à soma com 3 são selecionados (Figura 9).

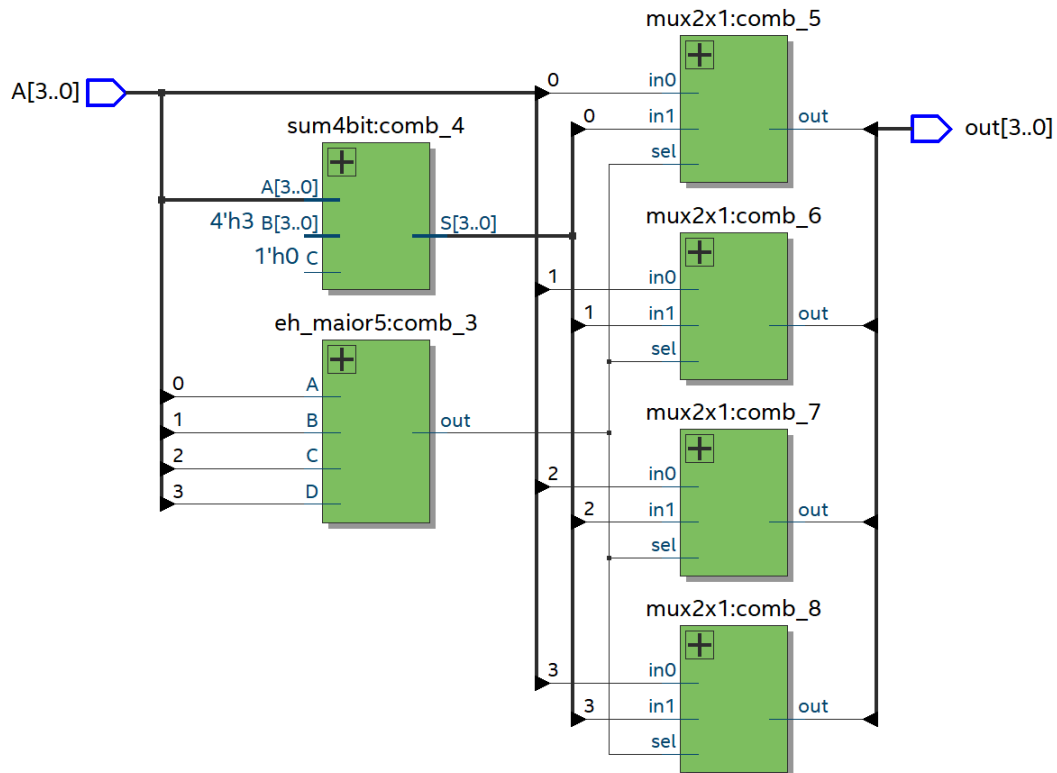


Figura 9. Implementação do double dabble em verilog
Fonte: Autoria própria

Dessa forma, o módulo `double_dabble` realiza a conversão binária para BCD, permitindo que o valor binário seja enviado para o módulo `decoder_decimal`, que, através da combinação de portas lógicas AND e OR, converte a palavra em BCD para valores binários que apresentam o número decimal no display.

O módulo principal `display_decimal` é o responsável por agrupar diversas instâncias do `double_dabble` e do `decoder_decimal` permitindo que um número de até 12 bits seja mostrado no display de 7 segmentos.

4.5.2 Decodificador hexadecimal

O módulo `display_hexa` foi desenvolvido para realizar a conversão e exibição de valores em formato hexadecimal nos displays de sete segmentos. Ele é responsável por receber três grupos de quatro bits, correspondentes às posições de unidade, dezena e centena, e gerar os sinais de controle necessários para acionar corretamente cada segmento dos três displays.

A conversão de cada grupo de 4 bits é feita por meio de instâncias do módulo `display_hex`. Esse módulo implementa a lógica combinacional que determina quais segmentos do display devem ser acesos para representar corretamente os dígitos de 0 a F em formato hexadecimal.

O funcionamento interno do `display_hex` baseia-se em combinações lógicas de portas AND, OR e NOT aplicadas às entradas `b0`, `b1`, `b2` e `b3`. Inicialmente, são gerados os sinais invertidos dessas entradas, permitindo a criação das condições necessárias para ativar cada

um dos sete segmentos (a, b, c, d, e, f e g) do display. Para cada segmento, um conjunto de expressões lógicas é utilizado a fim de cobrir todos os casos possíveis de ativação, de acordo com o valor binário fornecido.

Dessa forma, cada instância do módulo *display_hex* recebe um conjunto de 4 bits e gera os sete sinais de saída correspondentes aos segmentos do display. No módulo principal *display_hexa*, três instâncias são utilizadas, uma para cada dígito hexadecimal, permitindo que o circuito apresente números de até três dígitos em formato hexadecimal, com cada grupo de bits controlando independentemente um display.

4.5.3 Decodificador octal

O módulo *display_octal* foi desenvolvido para realizar a conversão e exibição de números na base octal em displays de sete segmentos. Ele é responsável por receber quatro grupos de três bits, correspondentes às posições de unidade, dezena, centena e milhar, e gerar os sinais necessários para acionar corretamente cada um dos quatro displays, permitindo a representação de números octais de até quatro dígitos.

A conversão de cada grupo de três bits é realizada por instâncias do módulo *d_octal*. Esse módulo implementa a lógica que define o estado de cada segmento (a, b, c, d, e, f e g) com base no valor binário de entrada, determinando a configuração necessária para representar corretamente os dígitos de 0 a 7 em formato octal.

O funcionamento interno do *d_octal* é baseado em combinações lógicas que utilizam portas NOT, AND, OR e XOR. Inicialmente, os sinais das entradas A, B e C são invertidos, possibilitando a criação das condições adequadas para ativação de cada segmento. Em seguida, operações XOR são utilizadas para identificar padrões binários específicos que correspondem aos números em octal. Cada segmento do display é então controlado por expressões lógicas próprias, formadas por diferentes combinações dessas portas, de modo a acender apenas os segmentos necessários para exibir o dígito correto.

No módulo principal *display_octal*, quatro instâncias do módulo *d_octal* são utilizadas uma para cada posição do número octal (unidade, dezena, centena e milhar). Cada instância recebe um grupo de três bits como entrada e produz sete sinais de saída que controlam diretamente um display de sete segmentos.

4.5.4 Módulo de seleção

O módulo *display7seg* foi desenvolvido para realizar a exibição de números nas bases decimal, hexadecimal e octal em quatro displays de sete segmentos. Ele recebe uma entrada de 16 bits (Data) que representa o valor a ser exibido e uma entrada de 2 bits (sel) que determina qual base numérica será apresentada. As saídas *d1*, *d2*, *d3* e *d4* correspondem aos sinais enviados para os quatro displays, possibilitando a visualização do número no formato selecionado.

A conversão para cada base é realizada por módulos específicos: *display_decimal*, *display_hexa* e *display_octal*. Cada um deles gera os sinais necessários para acionar corretamente os segmentos dos displays conforme a base correspondente. Esses módulos

funcionam de forma independente e têm suas saídas conectadas à lógica de seleção dentro do *display7seg*.

O funcionamento interno da lógica de seleção é baseado em combinações de portas NOT, AND e OR. Inicialmente, as portas NOT invertem os bits de seleção para permitir a identificação das três combinações válidas de base. Em seguida, as portas AND são utilizadas para ativar apenas uma base de exibição por vez, conforme o valor de *sel* decimal (11), hexadecimal (01) ou octal (10). Por fim, as portas OR combinam os sinais provenientes dos três módulos, garantindo que apenas os segmentos correspondentes à base ativa sejam exibidos nos displays.

4.6 Módulo principal

O módulo *main* foi projetado para integrar todos os módulos funcionais do circuito, garantindo que as operações aritméticas e lógicas aconteçam de forma coordenada e sincronizada com os sinais de controle. Ele recebe como entradas o dado principal *A* de 8 bits, o sinal de habilitação *Enable*, o clock *clk*, o seletor de operação *sel_ope* de 2 bits e o sinal *last_operation*, que indica se o valor da última operação deve ser reutilizado como entrada. A partir dessas entradas, o módulo organiza a execução de operações aritméticas, lógicas e de registro, direcionando os resultados para os displays e para as flags de controle.

O funcionamento do módulo começa pelo *mux_register*, que decide se a entrada da operação será o valor atual *A* ou o resultado armazenado da última operação *last_result*. Em seguida, o bloco de controle *top* realiza o debounce do sinal *Enable* e adiciona o valor de *A* na pilha digital, a última posição da pilha corresponde a *Data_A*, a segunda a *Data_B* e o topo da pilha ao seletor de operação *Ope*. Além disso, esse módulo gera o sinal de execução *executar*. As operações aritméticas são realizadas pelos módulos: *sum8bit* e *sub8bit_correto* efetuam soma e subtração, *multiplicador_alternativo* realiza a multiplicação de forma e *divisor8bit* realiza a divisão, enquanto os módulos lógicos *AND_operation*, *OR_operation*, *XOR_operation* e *NOT_operation* processam as operações lógicas correspondentes.

Os resultados de todas essas operações são enviados ao *mux_principal*, que seleciona o valor final a ser considerado de acordo com o tipo de operação definido por *Ope* e somente quando o sinal *executar* está ativo. Paralelamente, o *mux_de_flags* processa os sinais de carry, overflow, erro e zero, determinando quais flags devem ser ativadas com base nos resultados das operações e na execução do programa.

O módulo também inclui um *registrador do último resultado*, que armazena o valor da operação mais recente para que possa ser reutilizado em operações subsequentes. A atualização desse registrador é controlada por um pulso mais lento que teve origem no clock e foi modificado por 3 flip-flops, e do sinal de execução, garantindo que apenas valores válidos sejam armazenados. Por fim, os resultados selecionados são apresentados nos displays de 7 segmentos pelo módulo *display7seg*, que também determina em qual base numérica o resultado será mostrado, enquanto os sinais de overflow que excedem o tamanho padrão são detectados pelos módulos *overflow8bits* e *overflow12bits*.

Dessa forma, o módulo *main* atua como o centro de controle do circuito, coordenando a execução das operações, a seleção de resultados e a atualização de displays e flags,

garantindo que todos os blocos funcionais operem em conjunto de forma sincronizada e modular.

5. Resultados e Discussões

Para construir o circuito completo, foram elaboradas tabelas verdade correspondentes a cada módulo e operação implementada (soma, subtração, multiplicação, divisão, AND, OR, XOR e NOT). Essas tabelas permitiram verificar a consistência da lógica desenvolvida. Além disso, utilizou-se o Logisim[1] como ferramenta de apoio, possibilitando a simulação inicial da arquitetura da ULA e a detecção de eventuais inconsistências antes da etapa de síntese

Por fim, todos os módulos foram integrados em um projeto top-level hierárquico, simulados para verificação funcional e, posteriormente, programados na FPGA para validação prática na placa DE10-Lite, onde o funcionamento da ULA foi confirmado por meio da visualização dos resultados nos LEDs e displays de 7 segmentos.

5.1 Manual de Uso

Esta seção apresenta o modo de utilização do sistema desenvolvido para a placa FPGA DE10-Lite (10M50DAF484C7G), composta por 10 chaves (SW0–SW9) e 2 botões (KEY0 e KEY1). O controle das operações da ULA foi implementado seguindo a notação polonesa reversa (RPN), permitindo o empilhamento e processamento sequencial dos operandos.

O funcionamento ocorre da seguinte forma: inicialmente, o primeiro operando (A) é configurado por meio das chaves SW0 a SW7, que representam o valor binário de 8 bits. Em seguida, aciona-se a chave SW8, responsável por gerar o pulso de clock que armazena esse número na pilha digital. Depois, o segundo operando (B) é inserido da mesma maneira — utilizando novamente as chaves SW0 a SW7 — e confirmado com novo pulso de clock em SW8.

A operação desejada é então selecionada por meio das chaves SW0 a SW2, que definem o código binário correspondente à função aritmética ou lógica a ser executada. Após selecionar a operação, o usuário aciona mais uma vez SW8 para realizar o cálculo entre os dois operandos armazenados.

Quando se deseja reutilizar o resultado da operação anterior em um novo cálculo, a chave SW9 deve ser ativada no lugar do primeiro operando (A). Após isso, um novo valor B é configurado em SW0 a SW7, e o pulso de clock é novamente fornecido em SW8, reinserindo automaticamente o resultado anterior na pilha digital.

Por padrão, o resultado é exibido em base decimal. Caso o usuário deseje alterar a base de exibição, deve pressionar KEY1 para exibir o valor em base octal ou KEY0 para exibir o valor em base hexadecimal. A saída é apresentada nos displays de sete segmentos, enquanto as flags de Overflow, Carry Out, Zero e Erro são indicadas pelos LEDs da placa, sinalizando o estado atual do sistema.

5.2 Testes na FPGA

Após a etapa de simulação e verificação funcional no Quartus Prime, o circuito completo foi sintetizado e implementado na FPGA DE10-Lite (10M50DAF484C7G). Os testes práticos

foram conduzidos no Laboratório de Eletrônica Digital e Sistemas (LEDS), com o objetivo de comprovar o funcionamento correto de todas as operações aritméticas e lógicas da Unidade Lógica e Aritmética (ULA) desenvolvida.

O procedimento experimental seguiu rigorosamente a metodologia definida no *manual de uso*:

- Os operandos A e B foram inseridos por meio das chaves SW0 a SW7;
- O armazenamento dos valores foi feito acionando a chave SW8, responsável pelo pulso de clock;
- A seleção da operação foi realizada nas chaves SW0 a SW2;
- O resultado foi exibido nos displays de sete segmentos, inicialmente em base decimal;
- A alteração de base foi testada utilizando os botões KEY1 (octal) e KEY0 (hexadecimal).

Durante os testes, foram realizadas 18 operações distintas, abrangendo as funções aritméticas e lógicas especificadas no enunciado do problema. Cada teste foi devidamente documentado por meio de uma fotografia da placa FPGA em funcionamento, exibindo o resultado obtido e os indicadores luminosos correspondentes.

Em todos os experimentos, foi observada a coerência entre o valor esperado teoricamente e o valor apresentado pelo circuito físico, validando o correto comportamento do sistema em ambiente real.

A seguir, apresentam-se os registros de cada teste realizado, com suas respectivas configurações, resultados e observações.

1. Teste 1 – Soma ($A + B$)

Descrição: Teste da operação de soma binária entre dois operandos de 8 bits.

Configuração: $A = 1$, $B = 2$.

Resultado esperado: 3.

Resultado obtido: 3 (exibido no display, Figura 10).



Figura 10. Soma $A=1$ e $B=2$ resultando em 3

2. Teste 2 – Subtração ($A - B$)

Descrição: Teste da operação de subtração binária.

Configuração: $A = 7$, $B = 5$

Resultado esperado: 2

Resultado obtido: 2 (exibido no display, Figura 11)



Figura 11, Subtração $A=7$ e $B=5$ resultado em 2

3. Teste 3 – Multiplicação ($A \times B$)

Descrição: Teste da operação de multiplicação de 8 bits com saturação.

Configuração: $A = 255$, $B = 16$

Resultado esperado: 4080

Resultado obtido: 4080 (exibido no display, Figura 12)



Figura 12. Multiplicação $A= 255$ e $B= 16$

4. Teste 4 – Divisão ($A \div B$)

Descrição: Teste da operação de divisão com detecção de divisão por zero.

Configuração: $A = 127$, $B = 3$

Resultado esperado: 42

Resultado obtido: 42 (exibido no display, Figura 13)



Figura 13. Divisão $A =127$ e $B=3$ resultado em 42

5. Teste 5 – AND ($A \& B$)

Descrição: Teste da operação lógica AND bit a bit.

Configuração: A = 10(binário), B = 11(binário)

Resultado esperado: 2

Resultado obtido: 2 (exibido no display, Figura 14)



Figura 14. Operação AND

6. Teste 6 – OR ($A \mid B$)

Descrição: Teste da operação lógica OR bit a bit.

Configuração: A = 10 (binário), B 1010(binário)

Resultado esperado: 10

Resultado obtido: 10 (exibido no display, Figura 15)



Figura 15. Operação OR

7. Teste 7 – XOR ($A \oplus B$)

Descrição: Teste da operação lógica XOR bit a bit.

Configuração: A = 0011(binário), B = 1111(binário)

Resultado esperado: 12

Resultado obtido: 12 (exibido no display, Figura 16)



Figura 16. Operação XOR

8. Teste 8 – NOT ($\sim A$)

Descrição: Teste da operação de negação bit a bit.

Configuração: A = 00000000(binário)

Resultado esperado: 255

Resultado obtido: 255 (exibido no display, Figura 17)



Figura 17. Operação NOT

6. Conclusão

O resultado do sistema de Unidade Lógica Aritmética foi conduzido de forma sistemática, atendendo aos requisitos funcionais e operacionais estabelecidos. A implementação em Verilog estrutural permitiu organizar cada módulo de forma clara e modular, facilitando a integração entre as operações aritméticas e lógicas, bem como a sinalização das condições especiais por meio das *flags*.

Os testes realizados demonstraram que o circuito é capaz de executar corretamente as operações propostas (soma, subtração, multiplicação, divisão, AND, OR e XOR), além de identificar situações de erro, como divisão por zero e overflow. Dessa forma, o sistema mostrou-se consistente com a proposta inicial, validando a metodologia adotada desde a modelagem até a simulação.

Ainda assim, reconhece-se que o projeto pode ser expandido em versões futuras. Entre as possíveis melhorias, destacam-se: ampliar a faixa de exibição dos resultados, atualmente limitada a 99, para até 225; implementar a visualização dos resultados também em formato binário; e configurar o display de 7 segmentos para indicar o nome da operação realizada (MUL, DIV, SOM, SUB, XOR, AND e OR).

Em síntese, o desenvolvimento da ULA de 4 bits consolidou a aplicação prática dos conceitos de circuitos digitais e de linguagens de descrição de hardware, constituindo um importante exercício de integração entre teoria e prática no âmbito da engenharia da computação.

7. Referências

ABDELHADI, Ameer. Binary-to-BCD-Converter. 2019. Disponível em: <https://github.com/AmeerAbdelhadi/Binary-to-BCD-Converter>. (acessado em 20 de outubro de 2025).

FERNANDES, S. Conhecendo os Diferentes Tipos de Flip-Flops: RS, JK, D e T. Disponível em: <https://embarcados.com.br/conhecendo-os-diferentes-tipos-de-flip-flops-rs-jk-d-e-t/>. (acessado em 20 de outubro de 2025).

IDOETA, I. V. e CAPUANO, F. G. (2012) Elementos de Eletrônica Digital, 40ª ed., Editora Érica, São Paulo.

INTEL. DE10-Lite User Manual. Disponível em: https://ftp.intel.com/Public/Pub/fpgaup/pub/Intel_Material/Boards/DE10-Lite/DE10_Lite_User_Manual.pdf. (acessado em 20 de outubro de 2025).

MORENO, E. Organização e Arquitetura de Computadores – Aula 13. Disponível em: https://www.inf.pucrs.br/emoreno/undergraduate/CC/orgarqi/class_files/Aula13.pdf. (acessado em 20 de outubro de 2025).

TOCCI, R. J. (2011) Sistemas Digitais: Princípios e Aplicações, 10ª ed., Editora Pearson, Brasil.